Evaluating Snapshot-Consistent Live Migration for Stateful Containers in Kubernetes: A CRIU and Sidecar-Free eBPF Study

Musa Khasmammadov
Independent Systems Researcher
Baku, Azerbaijan
musaxasmammedov77@gmail.com

Abstract—Stateless microservices in Kubernetes tolerate node maintenance and cloud instance preemption through multi-replica rolling updates and PodDisruptionBudgets. Conversely, stateful singleton workloads that lack built-in distributed checkpointing—such as legacy numerical simulations, in-memory session aggregators, and warm machine-learning (ML) embedding caches—cannot be trivially replicated without consensus overhead or intrusive source-code refactoring. When an Amazon Web Services (AWS) EC2 Spot instance receives a two-minute interruption notice or a node is drained, the standard Kubernetes eviction sequence (SIGTERM followed by SIGKILL) destroys all volatile heap state.

We design, implement, and empirically evaluate an automated, snapshot-consistent live migration pipeline for stateful Kubernetes pods. Our system coordinates CRIU (v4.3) via the Kubelet Checkpoint API (KEP-2008), Buildah OCI layer packaging, the CRI-O runtime (v1.31), and a sidecar-free Cilium eBPF datapath. We evaluate the architecture on a three-node AWS EC2 cluster across four experimental dimensions: (1) Across $N = 36$ live migrations of a 26 MB stateful service compared against $N = 20$ native cold reschedules, decoupled checkpoint migration achieves a median cutover blackout of 1064.5 ms (mean 1132.4 ± 241.1 ms, minimum 797 ms) versus 1019.0 ms (mean 1007.0 ± 231.2 ms) for cold restart—adding only 45.5 ms (+4.5%) to median cutover latency and 144.0 ms of source freeze time while preserving in-memory state with a bounded Recovery Point Objective (RPO) of 669.0 ms (7.5 ± 0.6 ticks at 10 Hz). (2) Linear regression across all 36 runs ($r = 0.9624$) demonstrates that CRI-O unfreezes the restored container in ≈ 537 ms, whereas the remaining blackout variance arises from Kubelet's 1 s Pod Lifecycle Event Generator (PLEG) sync cycle. (3) Scaling an in-memory numerical weight buffer from 26 MB to 1024 MB (1 GB) with cryptographic SHA-256 heap verification confirms 100% bit-level state integrity across nodes within 50.09 s of source-side evacuation time (41.7% of the 120 s Spot deadline), while uncovering two systems bottlenecks: a Linux dirty-page write-back cliff above 512 MB and single-threaded gzip compression in Buildah (32.06 s at 1 GB). (4) An empirical comparison against a two-container loopback sidecar pod demonstrates why single-container KEP-2008 checkpointing breaks in-pod sidecar proxies but succeeds under host-level Cilium eBPF routing.

Index Terms—Kubernetes, Container Live Migration, CRIU, CRI-O, Cilium eBPF, Stateful Workloads, Spot Instances, Recovery Point Objective.

I. Introduction

Kubernetes schedules and heals workloads under the assumption that pod instances are stateless and interchangeable [1]. When a worker node undergoes kernel maintenance (kubectl drain) or an ephemeral cloud virtual machine is reclaimed, stateless deployments maintain availability by launching replacement replicas before terminating old pods (maxSurge).

However, many production workloads maintain large, mutable volatile heaps without native distributed persistence:

1) Single-Process Numerical and Scientific Runtimes: Iterative solvers, physics engines, and legacy C++/Python simulations that accumulate state in RAM over hours of execution.
2) Warm ML Inference Caches and Session Aggregators: In-memory embedding tables, KV caches, and stateful stream aggregators where rebuilding memory structures from cold object storage incurs severe warm-up penalties.

Running such workloads on Amazon Web Services (AWS) EC2 Spot instances reduces compute expenditure by 70–90% compared to On-Demand instances, but exposes pods to preemption with a hard 120-second termination notice (often preceded by an early EC2 Rebalance Recommendation) [2], [3]. During standard eviction, Kubelet sends SIGTERM, waits for terminationGracePeriodSeconds, and issues SIGKILL, wiping the process address space. Offloading state to an external key-value store such as Redis [4] avoids data loss during node preemption, but requires refactoring internal application data structures and replaces nanosecond-scale RAM references with sub-millisecond network serialization.

Userspace checkpoint/restore via CRIU [5] provides an application-transparent alternative [6], [7]. While Kubernetes v1.25+ introduced the Kubelet Checkpoint API (KEP-2008) primarily for forensic inspection [8], orchestrating KEP-2008 into a complete cross-node live migration pipeline exposes critical systems questions around downtime decomposition, state rollback, memory scaling, and service-mesh compatibility.

This paper makes four contributions:

- End-to-End Orchestration Pipeline: We build a privileged DaemonSet operator (PodLiveMigration CRD)

that automates KEP-2008 checkpointing, Buildah OCI packaging, registry transfer, CRI-O restoration, and Cilium eBPF routing across AWS EC2 nodes without application code changes.

- Statistical & Regression Analysis ($N = 36$ vs. $N = 20$): Comparing 36 CRIU migrations against 20 cold reschedules, we show that cutover blackout is governed in both cases by Kubelet's 1 s PLEG relist interval ($\sigma \approx 235$ ms). Regression of post-restore counter ticks against restore duration ($r = 0.9624$) isolates true container unfreeze time at ≈ 537 ms.
- Empirical Heap Scaling to 1 GB with SHA-256 Verification: We migrate dense numerical heaps from 26 MB to 1024 MB with 100% SHA-256 integrity, identifying the Linux vm.dirty_background_ratio writeback threshold and Buildah's single-threaded gzip step as the primary scaling bottlenecks.
- Empirical Sidecar vs. Sidecar-Free Evaluation: We experimentally demonstrate how KEP-2008's single-container checkpoint scope and tcp-close semantics sever in-pod sidecar loopback connections upon target restoration, validating the necessity of host-level eBPF service routing.

II. System Architecture

A. Migration Pipeline and CRIU Configuration

Our testbed runs on Ubuntu 24.04 LTS (Linux 6.8) with CRIU v4.3 [5], Kubelet v1.31.14 (KEP-2008 enabled) [8], CRI-O v1.31.4 configured with the runc runtime, Buildah, and Cilium v1.16.3 (kubeProxyReplacement=true). Figure 1 illustrates the nine-step migration workflow implemented in our operator (05-live-migration-operator.yaml).

Because a pod restored on a different worker node receives a new Pod IP from the target node's CIDR (10.244.2.0/24 → 10.244.1.0/24), /etc/criu/default.conf specifies five options [5]:

- tcp-close: Restores listening sockets (0.0.0.0:8090) in the LISTEN state while closing established TCP sockets bound to the old Pod IP.
- skip-in-flight: Drops transient in-flight TCP handshakes occurring during 50 Hz probing at the instant of ptrace seizure.
- skip-mnt /run/.containerenv: Skips CRI-O's transient metadata mount so CRImportCheckpoint mount validation succeeds.
- ext-unix-sk and shell-job: Allows dumping external UNIX sockets and inherited stdio descriptors.

B. Cutover Blackout Definition

Our benchmark service uses HTTP/1.0 (closing the TCP socket after each response) behind a Kubernetes Service ClusterIP and Cilium L7 Envoy Ingress without long-hold request buffering. Between source pod deletion (t_{kill}) and target endpoint readiness (t_4), incoming requests fail immediately with connection refused or HTTP 503. We define cutover blackout T_{blackout} strictly as the unavailability window during which probes fail, measured both by the operator clock ($t_4 - t_{\text{kill}}$) and by an external 50 Hz (± 20 ms) prober as the maximum interval between consecutive HTTP 200 OK responses.

III. Latency, State Drift, and Evacuation Model

A. Stage Timestamps and Disruption Decomposition

The operator records monotonic timestamps around each pipeline step: t_0 (before checkpoint API call), t_1 (after checkpoint archive creation), t_2 (after Buildah commit), t_3 (after registry push), t_{kill} (immediately after the pre-delete probe c_{kill} and just before DELETE /pods), and t_4 (upon the first HTTP 200 OK from the restored pod after Kubelet reports phase == Running):

$$T_{\text{freeze}} = t_1 - t_0, \quad T_{\text{build}} = t_2 - t_1, \quad T_{\text{push}} = t_3 - t_2 \tag{1}$$

$$T_{\text{restore}} = t_4 - t_3, \quad T_{\text{blackout}} = t_4 - t_{\text{kill}} = T_{\text{restore}} - t_{\text{probe}} \tag{2}$$

where $t_{\text{probe}} = t_{\text{kill}} - t_3 \approx 1$–$2$ ms is the latency of the pre-delete HTTP GET used to sample c_{kill}. Thus T_{blackout} is deterministically 1–2 ms shorter than T_{restore}.

Client-perceived disruption consists of the source-side freeze stutter T_{freeze} (while threads are paused by ptrace and incoming packets wait in the kernel TCP backlog) plus the cutover blackout T_{blackout}:

$$T_{\text{disruption}} = T_{\text{freeze}} + T_{\text{blackout}} \tag{3}$$

$$T_{\text{total}} = T_{\text{freeze}} + T_{\text{build}} + T_{\text{push}} + T_{\text{restore}} \tag{4}$$

B. State Drift (RPO), Post-Restore Ticks, and Rollback

In Mode A (Decoupled, KeepRunning: true), CRI-O unfreezes the source container at t_1, allowing it to serve traffic while Buildah packages and pushes the image. Any state mutations occurring between the snapshot ($S_{\text{ckpt}} \approx c_{\text{dump}} + 1$) and pod deletion ($t_{\text{kill}}$) are lost upon cutover. At mutation rate λ, the Recovery Point Objective (RPO) and lost update count ΔS_{lost} are:

$$\text{RPO} = T_{\text{build}} + T_{\text{push}} \tag{5}$$

$$\Delta S_{\text{lost}} = c_{\text{kill}} - S_{\text{ckpt}} \approx \lambda \cdot (T_{\text{build}} + T_{\text{push}}) \tag{6}$$

Because clients querying the source pod between t_1 and t_{kill} observe state values up to c_{kill} that revert to S_{ckpt} after cutover, workloads with external side effects require idempotency keys or a write-ahead log (WAL).

In Mode B (Synchronous Stop-and-Copy), terminating or halting the source container immediately at t_1 yields RPO $\equiv 0$, but extends service unavailability to the entire pipeline ($T_{\text{blackout}}^{\text{stop-and-copy}} = T_{\text{total}}$).

Crucially, on the target node, runc restore unfreezes the container at time $T_{\text{unfreeze}} < T_{\text{restore}}$, and the restored process immediately resumes incrementing from S_{ckpt} at rate λ. However, the operator only probes c_{restore} after Kubelet's PLEG loop (1 s relist period) updates Pod.status.phase = Running at $T_{\text{restore}} = T_{\text{unfreeze}} +$

Nine-Stage Decoupled Container Live Migration Pipeline:
1. Hazard Trigger (T_{detect}): Operator detects node maintenance (cordon), Spot Interruption Notice, or manual CRD creation.
2. Pre-Dump Probe & Checkpoint Freeze (T_{freeze}): Operator reads pre-dump counter c_{dump} and invokes POST :10250/checkpoint/{ns}/{pod}/{container}. CRIU halts threads via ptrace(PTRACE_SEIZE), writes memory pages to checkpoint.tar, and unfreezes the source container (KeepRunning: true).
3. OCI Layer Packaging (T_{build}): Buildah packages checkpoint.tar into a compressed OCI image annotated with io.kubernetes.cri-o.annotations.checkpoint.name.
4. Registry Push (T_{push}): Buildah pushes the image to the in-cluster registry (10.10.1.10:5000). Source-side evacuation is now complete.
5. Pre-Kill Probe & Source Deletion (t_{kill}): Operator records c_{kill} and issues DELETE /pods/{pod}?gracePeriodSeconds=0; cutover blackout (T_{blackout}) begins.
6. Target Pod Creation: Operator creates the target Pod manifest pinned to the destination worker via spec.nodeName.
7. CRI-O Pull & Restore ($T_{\text{unfreeze}} \approx 537$ ms): Target CRI-O pulls the checkpoint layer, verifies identical bind mounts (automountServiceAccountToken: false), and invokes runc/criu restore.
8. Process Resumption: Target process re-binds 0.0.0.0:8090 in its new Pod network namespace and immediately resumes execution from the checkpointed state.
9. PLEG Sync & Readiness Probe (T_{restore}): Kubelet PLEG updates Pod.status.phase = Running in kube-apiserver; operator verifies HTTP 200 OK on the new Pod IP and records c_{restore}.

Fig. 1. Execution sequence of the decoupled snapshot-consistent live migration pipeline.

Δt_{pleg}. During Δt_{pleg}, the restored process accumulates $\Delta S_{\text{post-restore}}$ new ticks:

$$c_{\text{restore}} - c_{\text{dump}} \approx \lambda \cdot (T_{\text{restore}} - T_{\text{unfreeze}}) \tag{7}$$

C. Source-Side Spot Evacuation Condition

When AWS issues a 120-second Spot Interruption Notice, only the source node is terminated at $t = 120$ s; target-side pull and restoration (T_{restore}) execute on the surviving destination node after t_3. Therefore, Spot survival requires only that source-side evacuation $T_{\text{evac}}(M)$ complete within 120 s:

$$\begin{aligned} T_{\text{evac}}(M) = {} & T_{\text{detect}} + T_{\text{freeze}}(M) \\ & + T_{\text{build}}(M) + T_{\text{push}}(M) < 120 \text{ s} \end{aligned} \tag{8}$$

where $T_{\text{detect}} \approx 2.5$ s for a 5 s IMDSv2 poll interval.

IV. Empirical Evaluation

A. Testbed Setup

Experiments were conducted on a three-node AWS EC2 cluster in eu-central-1: one c7i-flex.large control-plane node (2 vCPU, 4 GB RAM, hosting the OCI registry at 10.10.1.10:5000 and Cilium Ingress) and two m7i-flex.large workers (2 vCPU, 8 GB RAM, EBS gp3 storage). All 11 microservices of the Google Online Boutique suite ran continuously across the workers to generate background multi-tenant CPU, memory, and CNI contention.

B. Experiment 1: Statistical Distribution ($N = 36$ CRIU vs. $N = 20$ Cold Baseline)

Table I summarizes the empirical distribution across $N = 36$ CRIU migrations ($n = 20$ from worker-1 to worker-2; $n = 16$ from worker-2 to worker-1) and $N = 20$ native cold reschedules of the 26 MB stateful counter service ($\lambda = 10$ Hz). Table II compares two individual UI traces (#35 and #36) and Mode B against the cold baseline.

Three key findings emerge from Table I and Table II:

1) Negligible Cutover Overhead vs. Cold Startup: Across $N = 36$ CRIU runs and $N = 20$ cold reschedules, median cutover blackout is 1064.5 ms for CRIU versus 1019.0 ms for cold startup—an overhead of only +45.5 ms (+4.5%) to pull the 5.16 MiB checkpoint layer and restore memory pages via CRIU, while preserving the process heap instead of resetting it to zero.
2) Kubelet PLEG Dominance and Unfreeze Regression: Both CRIU restore ($\sigma = 241.1$ ms) and cold startup ($\sigma = 231.2$ ms) exhibit nearly identical standard deviations, matching the theoretical standard deviation of a uniform 1 s Kubelet PLEG relist cycle ($1000/\sqrt{12} = 288.7$ ms). Ordinary least-squares regression of ($c_{\text{restore}} - c_{\text{dump}}$) against T_{restore} across all 36 CRIU runs yields Pearson correlation $r = 0.9624$, slope $\hat{\lambda} = 9.96$ ticks/s (matching the 10.0 Hz counter), and x-intercept $T_{\text{unfreeze}} = 537.1$ ms. Thus, CRI-O actually unfreezes the restored container in ≈ 537 ms; the remaining 261–1151 ms of T_{restore} is Kubelet PLEG status synchronization delay (Δt_{pleg}).
3) Resolution of Trace #35 vs. Trace #36: Across all 36 runs, $c_{\text{kill}} - c_{\text{dump}}$ is tightly bounded to 7.5 ± 0.6 ticks (range 7–9), confirming Eq. (6). In Trace #35 ($51 \to 60 \to 57$, $T_{\text{restore}} = 1080$ ms), the restored container ticked ≈ 5 times during its 543 ms PLEG wait ($1080 - 537$ ms), whereas in Trace #36 ($77 \to$

TABLE I
Statistical Comparison: $N = 36$ CRIU Live Migrations vs. $N = 20$ Native Cold Reschedules (26 MB Service)

Configuration / Subset	T_{freeze} (ms)	T_{build} (ms)	T_{push} (ms)	RPO (ms)	T_{restore} (ms)	T_{blackout} (ms)	$T_{\text{disruption}}$ (ms)	T_{total} (ms)	$c_{\text{kill}} - c_{\text{dump}}$ (ticks)
CRIU w1 → w2 ($n = 20$)									
Median ($p50$) / $p95$	144.0 / 158	368.5 / 400	293.0 / 310	664.0 / 703	1033.0 / 1533	1031.5 / 1532	1178.0 / 1673	1857.0 / 2358	7.5 / 8
Mean ± SD [Min–Max]	143.3 ± 9.4	370.9 ± 17.1	293.6 ± 12.5	664.6 ± 27.6	1112.9 ± 249.2	1111.2 ± 249.3 [817–1687]	1254.5 ± 250.1 [958–1836]	1920.8 ± 248.7	7.6 ± 0.6 [7–9]
CRIU w2 → w1 ($n = 16$)									
Median ($p50$) / $p95$	145.0 / 155	372.5 / 384	301.5 / 308	672.5 / 688	1092.0 / 1474	1090.5 / 1472	1229.0 / 1622	1887.0 / 2277	7.0 / 8
Mean ± SD [Min–Max]	145.4 ± 7.1	366.6 ± 16.4	296.6 ± 12.4	663.2 ± 27.9	1160.2 ± 235.9	1158.8 ± 235.7 [797–1682]	1304.1 ± 238.1 [944–1828]	1968.8 ± 238.0	7.4 ± 0.5 [7–8]
CRIU Combined ($N = 36$)									
Median ($p50$) / $p95$	144.0 / 156	370.5 / 389	298.5 / 310	669.0 / 696	1066.0 / 1533	1064.5 / 1532	1203.0 / 1673	1878.0 / 2358	7.0 / 8 (RAM Kept)
Mean ± SD [Min–Max]	144.2 ± 8.4	369.0 ± 16.7	294.9 ± 12.4	663.9 ± 27.4	1133.9 ± 241.1	1132.4 ± 241.1 [797–1687]	1276.6 ± 242.6 [944–1836]	1942.1 ± 241.8	7.5 ± 0.6 [7–9]
Native Cold Baseline ($N = 20$)									
Median ($p50$) / $p95$	N/A	N/A	N/A	Total Loss	1019.0 / 1244	1019.0 / 1244	1019.0 / 1244	1019.0 / 1244	100% State Lost
Mean ± SD [Min–Max]	N/A	N/A	N/A	Total Loss	1007.0 ± 231.2	1007.0 ± 231.2 [712–1312]	1007.0 ± 231.2 [712–1312]	1007.0 ± 231.2	Reset to 0 [1–6]

TABLE II
Detailed Trace Breakdown and Operating Mode Trade-offs (RTO vs. RPO)

Mode / Trace ID	Path	T_{total}	T_{freeze}	T_{build}	T_{push}	T_{blackout}	$T_{\text{disruption}}$	RPO	Counter($c_{\text{dump}} \to c_{\text{kill}} \to c_{\text{restore}}$)
Cold Reschedule (UI Trace)	w2 → w1	1289 ms	N/A	N/A	N/A	1289 ms	1289 ms	Total Loss	43544 → 8 (Lost 43536 ticks)
CRIU Decoupled (Trace #35)	w1 → w2	1935 ms	159 ms	385 ms	310 ms	1078 ms	1237 ms	695 ms	51 → 60 → 57 ($\Delta S_{\text{lost}} \approx 8, \Delta S_{\text{post}} \approx 5$)
CRIU Decoupled (Trace #36)	w2 → w1	1642 ms	147 ms	388 ms	308 ms	797 ms	944 ms	696 ms	77 → 85 → 79 ($\Delta S_{\text{lost}} \approx 7, \Delta S_{\text{post}} \approx 1$)
Stop-and-Copy (Mode B, Med.)	Bidirectional	1878 ms	144 ms	370.5 ms	298.5 ms	1878 ms	1878 ms	0 ms	Zero Drift ($T_{\text{blackout}} = T_{\text{total}}$)

85 → 79, $T_{\text{restore}} = 798$ ms), the 261 ms PLEG wait produced only ≈ 1–2 post-restore ticks.

C. Experiment 2: Empirical Heap Scaling (26 MB to 1024 MB) with SHA-256 Verification

To evaluate realistic stateful workloads beyond a small counter, we implemented an in-memory numerical state workload that populates a contiguous heap buffer of $M \in \{64, 128, 256, 512, 1024\}$ MB with high-entropy pseudo-random float64 weight blocks (simulating an in-memory ML embedding cache or scientific grid). Before checkpointing on k8s-worker-1, the workload computes a SHA-256 digest of the entire heap buffer; after restoration on k8s-worker-2, the /live/verify endpoint recomputes the SHA-256 digest over the restored memory. Table III reports the empirical measurements.

Table III reveals three important systems properties:

1) Bit-Exact Restoration up to 1 GB within Spot SLA: All heap sizes up to 1024 MB restored with 100% SHA-256 verification. At 1 GB, source-side evacuation ($T_{\text{freeze}} + T_{\text{build}} + T_{\text{push}}$) completed in 50.09 s (41.7% of the 120 s AWS Spot budget), confirming that 1–2 GB containers can be evacuated before Spot reclamation.
2) The Kernel Dirty-Page Cache Cliff in T_{freeze}: From 64 MB to 512 MB, T_{freeze} remains sub-second (216 ms → 760 ms, effective write rate ≈ 680 MiB/s) because Kubelet writes checkpoint.tar into the Linux page cache. Between 512 MB and 1024 MB, the checkpoint crosses the kernel's vm.dirty_background_ratio threshold (≈ 780 MB on an 8 GB node), forcing synchronous writeback to EBS gp3 (≈ 138 MiB/s) and increasing T_{freeze} by 9.8× to 7.464 s.
3) Buildah Single-Threaded gzip Bottleneck: At 1 GB, T_{build} consumes 32.060 s (64.0% of source evacuation time and 75.2% of the 42.630 s RPO drift, where $\Delta S_{\text{lost}} = 447 - 19 = 428$ ticks matches $\lambda \cdot \text{RPO} = 426.3$ ticks). Inspection confirms that buildah commit compresses layers using single-threaded gzip (≈ 32.1 MiB/s). Disabling compression or using multi-threaded zstd for cluster-local registries is a high-leverage optimization.

D. Experiment 3: In-Pod Sidecar Proxy vs. Sidecar-Free Cilium eBPF

To test service-mesh compatibility empirically, we deployed a two-container pod (bench-sidecar-pod) on k8s-worker-1 comprising stream-server (listening on 127.0.0.1:8090) and an envoy-sidecar proxy (listening on 0.0.0.0:15006 and forwarding over a persistent loopback TCP socket to 127.0.0.1:8090). Prior to checkpointing, probes to :15006 returned HTTP 200 OK (ram_counter = 505).

When triggered via PodLiveMigration, KEP-2008 dumped stream-server in 118 ms ($T_{\text{build}} = 266$ ms, $T_{\text{push}} = 180$ ms), and CRI-O restored the container on k8s-worker-2. However, because KEP-2008 checkpoints only a single container per invocation and tcp-close closes the established 127.0.0.1:8090 loopback socket upon restoration, the restored backend was permanently isolated in the target pod namespace ($T_{\text{blackout}} = \infty$, timing out after 120 s). By contrast, Cilium's sidecar-free eBPF datapath maintains L4/L7 routing in the host kernel outside the container namespace, allowing all 36 single-container migrations to succeed without sidecar re-peering.

V. Related Work

Clark et al. [10] established iterative pre-copy live migration for virtual machines. For Linux containers, Stoyanov and Kollingbaum [6] reduced CRIU filesystem transfer overhead using an image cache proxy, and Nadgowda et al. [7] developed Voyager to coordinate CRIU memory dumps

TABLE III
Empirical Memory Scaling on AWS EC2 (26 MB to 1024 MB Heap) with Cross-Node SHA-256 Heap Integrity Verification

Allocated Heap	OCI Layer Size	T_{freeze} (Dump)	T_{build} (Buildah)	T_{push} (Registry)	RPO (Δt_{drift})	Source T_{evac} (% 120s SLA)	T_{restore}	T_{total}	Restored SHA-256 Integrity
26 MB ($N = 36$ Med.)	5.16 MiB	0.144 s	0.371 s	0.299 s	0.669 s (7 ticks)	0.81 s (0.7%)	1.066 s	1.878 s	Verified (Counter state)
64 MB (Weights)	69.11 MiB	0.216 s	0.761 s	0.615 s	1.376 s (14 ticks)	1.59 s (1.3%)	0.951 s	2.543 s	100% Match (7785bfc701411dcb)
128 MB (Weights)	133.11 MiB	0.258 s	1.082 s	0.929 s	2.011 s (21 ticks)	2.27 s (1.9%)	1.695 s	3.964 s	100% Match (cce52f481cd5ca56)
256 MB (Weights)	261.13 MiB	0.375 s	5.796 s	1.685 s	7.481 s (75 ticks)	7.86 s (6.6%)	2.108 s	9.964 s	100% Match (4b1b1359a0222290)
512 MB (Weights)	517.15 MiB	0.760 s	16.120 s	3.267 s	19.387 s (195 ticks)	20.15 s (16.8%)	7.807 s	27.954 s	100% Match (ab028a06dcdc25bf)
1024 MB (1 GB)	1029.21 MiB	7.464 s	32.060 s	10.570 s	42.630 s (428 ticks)	50.09 s (41.7%)	17.328 s	67.422 s	100% Match (c0da7edfafcf15a6)

with union storage federation. Reber [8] introduced forensic container checkpointing into Kubernetes via KEP-2008. Our work extends KEP-2008 into an automated cross-node migration controller on CRI-O and Cilium eBPF [9], providing empirical characterization of Kubelet PLEG latency, page-cache writeback cliffs, Buildah compression bottlenecks, and sidecar incompatibility.

VI. Limitations and Future Work

First, our workload uses HTTP/1.0 with CRIU's tcp-close option; migrating persistent HTTP/1.1 keep-alive, WebSocket, or UDP flows requires L7 ingress retry buffering or CRIU –tcp-established with static Pod IP routing. Second, for heaps ≥ 256 MB, single-threaded gzip in Buildah inflates RPO (7.48–42.63 s); replacing OCI tar/gzip packaging with direct criu page-server memory streaming and iterative pre-dumping (criu dump –pre-dump) is the natural next step to achieve sub-second freeze and RPO on multi-gigabyte workloads.

VII. Conclusion

We presented an empirical study of snapshot-consistent container live migration in Kubernetes using CRI-O, CRIU v4.3, Buildah, and sidecar-free Cilium eBPF. Across $N = 36$ migrations and $N = 20$ cold reschedules on AWS EC2, CRIU restoration adds only 45.5 ms (+4.5%) to median cutover blackout (1064.5 ms vs. 1019.0 ms) while preserving 100% of snapshot heap state across footprints up to 1 GB. Regression and scaling analyses identify Kubelet's 1 s PLEG loop, the Linux dirty-page writeback threshold, and Buildah gzip compression as the primary targets for future runtime optimization.

Artifact Availability

All manifests, Ansible playbooks, operator code, and benchmark harnesses are available at: https://github.com/musaxasmammedov7/final-research.

References

[1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016.

[2] Netflix Technology Blog, "Creating Your Own EC2 Spot Market," Netflix Inc., 2015.

[3] Amazon Web Services, "Spot Instance Interruptions and EC2 Rebalance Recommendations," AWS Documentation, 2024.

[4] Redis Ltd., "Redis Latency Problems Troubleshooting," https://redis.io/docs/management/optimization/latency/, 2024.

[5] Checkpoint/Restore In Userspace, "CRIU CLI Options and Architecture," https://criu.org, 2024.

[6] R. Stoyanov and M. J. Kollingbaum, "Efficient Live Migration of Linux Containers," in High Performance Computing: ISC High Performance 2018 International Workshops, LNCS, vol. 11203, Springer, Cham, 2018, pp. 184–193.

[7] S. Nadgowda, S. Suneja, N. Bila, and C. Isci, "Voyager: Complete Container State Migration," in Proc. IEEE 37th Int. Conf. on Distributed Computing Systems (ICDCS), 2017, pp. 2137–2142.

[8] A. Reber, "Forensic Container Checkpointing in Kubernetes," Kubernetes Official Blog / CNCF, Dec. 2022. [Online]. Available: https://kubernetes.io/blog/2022/12/05/forensic-container-checkpointing-alpha/

[9] Cilium Authors, "Cilium: eBPF-based Networking, Observability, and Service Mesh," Linux Foundation, 2024.

[10] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live Migration of Virtual Machines," in Proc. 2nd USENIX Symp. on Networked Systems Design and Implementation (NSDI), 2005, pp. 273–286.